

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №4.

Тема: «

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Базовый класс:

ТРОЙКА_ЧИСЕЛ (TRIAD)

- Первое_число (first) – int
- Второе_число (second) – int
- Третье_число (third) – int

Определить методы изменения полей и увеличения полей на 1.

Создать производный класс **DATE** с полями год, месяц и число.

Переопределить методы увеличения полей на 1 и определить метод увеличения даты на n дней.

Анализ задачи

1. Программа строится вокруг двух классов: базового TRIAD и производного DATE.
2. Базовый класс TRIAD просто хранит три числа и позволяет ими управлять: читать, изменять, увеличивать по одному или все сразу, без каких-либо дополнительных правил.
3. Производный класс DATE берёт тройку чисел и трактует их как день, месяц и год, добавляя календарные правила.
4. Календарные правила включают учёт количества дней в разных месяцах, определение високосных годов и автоматическое исправление некорректных дат.
5. Поскольку в DATE появляются собственные поля (день, месяц, год), необходимо постоянно согласовывать их с унаследованными полями (first, second, third), обновляя последние при любом изменении даты.
6. Ключевой механизм исправления даты работает так: если день превышает допустимое количество дней в текущем месяце, лишние дни переносятся в следующий месяц, а при переполнении месяца — в следующий год.
7. Метод увеличения на один день (incAll) в базовом классе увеличивает все три числа, а в производном классе увеличивает только день, оставляя остальную коррекцию календарю.

8. Добавление нескольких дней реализовано через многократный вызов метода увеличения на один день.

9. В главной функции демонстрируется создание объектов разными конструкторами, копирование, присваивание, ввод и вывод данных, работа методов увеличения и автоматическая коррекция неверных дат.

10. Также демонстрируется, что объект DATE можно использовать везде, где ожидается объект TRIAD.

11. Базовый класс задаёт общую структуру и интерфейс, а производный класс наполняет этот интерфейс конкретным календарным смыслом, изменяя поведение ключевых методов.

Код

```
#include <iostream>
#include <locale>

using namespace std;

class TRIAD {
protected:
    int first;
    int second;
    int third;

public:
    TRIAD() : first(0), second(0), third(0) {
        cout << "Конструктор TRIAD по умолчанию\n";
    }

    TRIAD(int f, int s, int t) : first(f), second(s), third(t) {
        cout << "Конструктор TRIAD с параметрами\n";
    }

    TRIAD(const TRIAD& other) : first(other.first), second(other.second),
third(other.third) {
        cout << "Конструктор копирования TRIAD\n";
    }

    virtual ~TRIAD() {
        cout << "Деструктор TRIAD\n";
    }

    int getFirst() const { return first; }
    int getSecond() const { return second; }
    int getThird() const { return third; }

    void setFirst(int f) { first = f; }
    void setSecond(int s) { second = s; }
    void setThird(int t) { third = t; }

    void incFirst() { first++; }
    void incSecond() { second++; }
    void incThird() { third++; }

    virtual void incAll() {
        incFirst();
        incSecond();
        incThird();
    }
}
```

```

    TRIAD& operator=(const TRIAD& other) {
        cout << "Оператор присваивания TRIAD\n";
        if (this != &other) {
            first = other.first;
            second = other.second;
            third = other.third;
        }
        return *this;
    }

    friend ostream& operator<<(ostream& os, const TRIAD& t);
    friend istream& operator>>(istream& is, TRIAD& t);
};

ostream& operator<<(ostream& os, const TRIAD& t) {
    os << "(" << t.first << ", " << t.second << ", " << t.third << ")";
    return os;
}

istream& operator>>(istream& is, TRIAD& t) {
    cout << "Введите три числа: ";
    is >> t.first >> t.second >> t.third;
    return is;
}

class DATE : public TRIAD {
private:
    int day;
    int month;
    int year;

    int daysInMonth(int m, int y) {
        if (m == 2) {
            if ((y % 4 == 0 && y % 100 != 0) || (y % 400 == 0))
                return 29;
            return 28;
        }
        if (m == 4 || m == 6 || m == 9 || m == 11) return 30;
        return 31;
    }

    void fixDate() {
        while (day > daysInMonth(month, year)) {
            day -= daysInMonth(month, year);
            month++;
            if (month > 12) {
                month = 1;
                year++;
            }
        }
        updateFields();
    }

    void updateFields() {
        first = day;
        second = month;
        third = year;
    }

public:
    DATE() : TRIAD(), day(1), month(1), year(2000) {
        cout << "Конструктор DATE по умолчанию\n";
        updateFields();
    }

    DATE(int d, int m, int y) : TRIAD(), day(d), month(m), year(y) {
        cout << "Конструктор DATE с параметрами\n";
        updateFields();
    }
};

```

```

    }

    DATE(const DATE& other) : TRIAD(other), day(other.day), month(other.month),
year(other.year) {
        cout << "Конструктор копирования DATE\n";
        updateFields();
    }

    ~DATE() {
        cout << "Деструктор DATE\n";
    }

    int getDay() const { return day; }
    int getMonth() const { return month; }
    int getYear() const { return year; }

    void setDay(int d) { day = d; fixDate(); }
    void setMonth(int m) { month = m; fixDate(); }
    void setYear(int y) { year = y; fixDate(); }

    void incAll() override {
        day++;
        fixDate();
    }

    void addDays(int n) {
        for (int i = 0; i < n; i++) {
            incAll();
        }
    }

    DATE& operator=(const DATE& other) {
        cout << "Оператор присваивания DATE\n";
        if (this != &other) {
            TRIAD::operator=(other);
            day = other.day;
            month = other.month;
            year = other.year;
            updateFields();
        }
        return *this;
    }

    friend ostream& operator<<(ostream& os, const DATE& d);
    friend istream& operator>>(istream& is, DATE& d);
};

ostream& operator<<(ostream& os, const DATE& d) {
    os << d.day << "." << d.month << "." << d.year;
    return os;
}

istream& operator>>(istream& is, DATE& d) {
    cout << "Введите день месяц год: ";
    is >> d.day >> d.month >> d.year;
    d.updateFields();
    return is;
}

void printObject(TRIAD& obj) {
    cout << "Объект: " << obj << endl;
}

TRIAD createObject(int a, int b, int c) {
    TRIAD temp(a, b, c);
    return temp;
}

int main() {

```

```

setlocale(LC_ALL, "Russian");

cout << "ДЕМОНСТРАЦИЯ РАБОТЫ КЛАССА TRIAD \n\n";

TRIAD t1;
cout << "t1: " << t1 << endl;

TRIAD t2(10, 20, 30);
cout << "t2: " << t2 << endl;

TRIAD t3(t2);
cout << "t3 (копия t2): " << t3 << endl;

t1 = t2;
cout << "t1 после присваивания: " << t1 << endl;

cout << "\nВведите данные для t4:\n";
TRIAD t4;
cin >> t4;
cout << "t4: " << t4 << endl;

t2.incFirst();
t2.incSecond();
t2.incThird();
cout << "\nt2 после увеличения каждого поля: " << t2 << endl;

t2.incAll();
cout << "t2 после incAll(): " << t2 << endl;

cout << "\nДЕМОНСТРАЦИЯ РАБОТЫ КЛАССА DATE \n\n";

DATE d1;
cout << "d1: " << d1 << endl;

DATE d2(28, 2, 2024);
cout << "d2: " << d2 << endl;

DATE d3(d2);
cout << "d3 (копия d2): " << d3 << endl;

d1 = d2;
cout << "d1 после присваивания: " << d1 << endl;

cout << "\nУвеличиваем d2 на 1 день: ";
d2.incAll();
cout << d2 << endl;

cout << "Увеличиваем d2 на 3 дня: ";
d2.addDays(3);
cout << d2 << endl;

cout << "\nВведите данные для d4:\n";
DATE d4;
cin >> d4;
cout << "d4: " << d4 << endl;

d4.setDay(15);
cout << "\nПосле setDay(15): " << d4 << endl;

cout << "\nПРИНЦИП ПОДСТАНОВКИ \n\n";
cout << "Вызов printObject с TRIAD: ";
printObject(t1);
cout << "Вызов printObject с DATE: ";
printObject(d1);

TRIAD t5 = createObject(100, 200, 300);
cout << "t5 из createObject: " << t5 << endl;

cout << "\nПрограмма завершена\n";

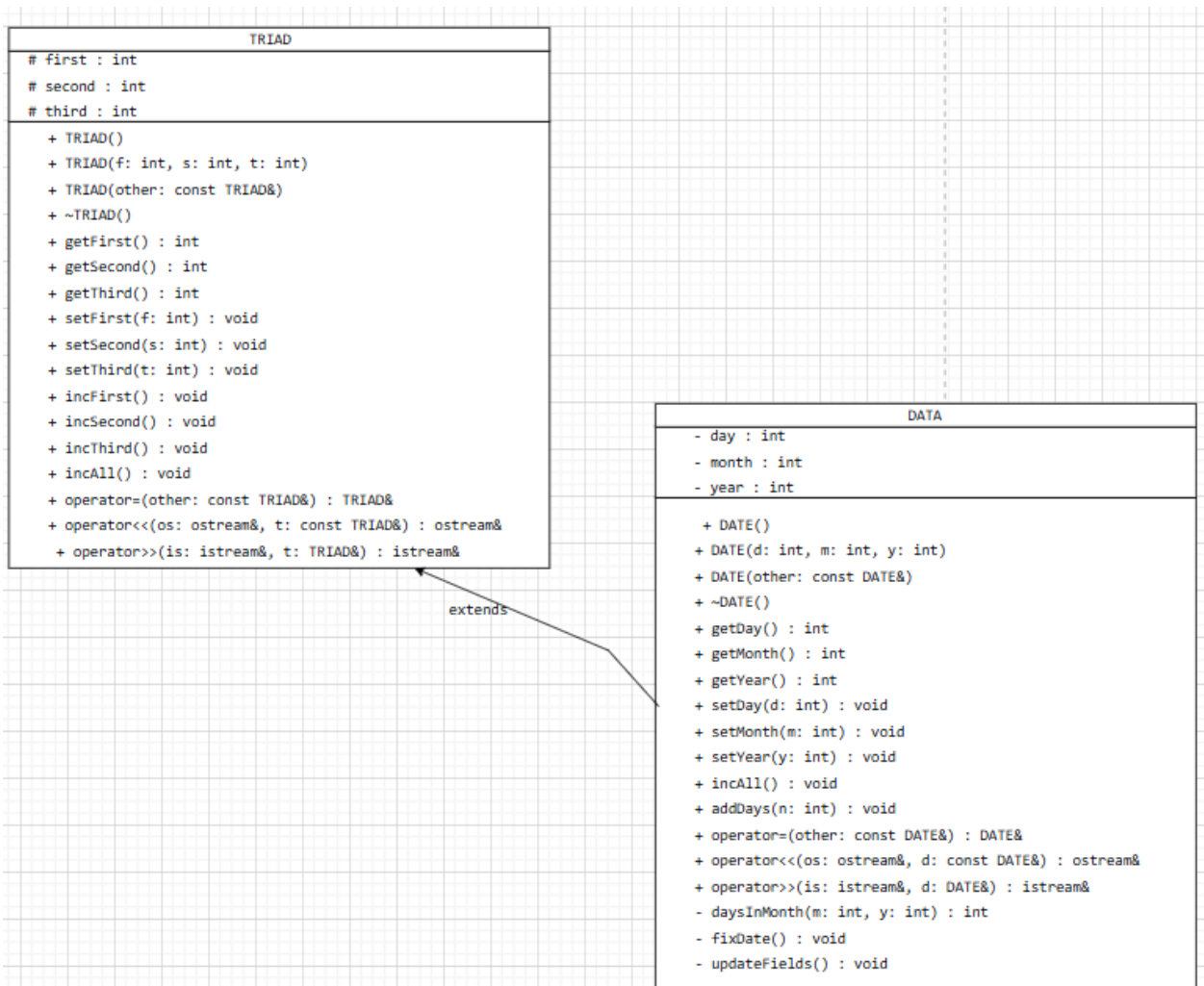
```

```

    return 0;
}

```

UML диаграмма



Вывод программы

ДЕМОНСТРАЦИЯ РАБОТЫ КЛАССА TRIAD

```

Конструктор TRIAD по умолчанию
t1: (0, 0, 0)
Конструктор TRIAD с параметрами
t2: (10, 20, 30)
Конструктор копирования TRIAD
t3 (копия t2): (10, 20, 30)
Оператор присваивания TRIAD
t1 после присваивания: (10, 20, 30)

Введите данные для t4:
Конструктор TRIAD по умолчанию
Введите три числа: 5 6 7
t4: (5, 6, 7)

t2 после увеличения каждого поля: (11, 21, 31)
t2 после incAll(): (12, 22, 32)

```

ДЕМОНСТРАЦИЯ РАБОТЫ КЛАССА DATE

```

Конструктор TRIAD по умолчанию
Конструктор DATE по умолчанию
d1: 1.1.2000
Конструктор TRIAD по умолчанию
Конструктор DATE с параметрами
d2: 28.2.2024
Конструктор копирования TRIAD
Конструктор копирования DATE
d3 (копия d2): 28.2.2024
Оператор присваивания DATE
Оператор присваивания TRIAD
d1 после присваивания: 28.2.2024

Увеличиваем d2 на 1 день: 29.2.2024
Увеличиваем d2 на 3 дня: 3.3.2024

Введите данные для d4:
Конструктор TRIAD по умолчанию
Конструктор DATE по умолчанию
Введите день месяц год: 3 5 2021
d4: 3.5.2021

После setDay(15): 15.5.2021

```

ПРИНЦИП ПОДСТАНОВКИ

```

Вызов printObject с TRIAD: Объект: (10, 20, 30)
Вызов printObject с DATE: Объект: (28, 2, 2024)
Конструктор TRIAD с параметрами
t5 из createObject: (100, 200, 300)

```

Программа завершена

Деструктор TRIAD

Деструктор DATE

Деструктор TRIAD

Деструктор DATE

Деструктор TRIAD

Деструктор DATE

Деструктор TRIAD

Деструктор DATE

Деструктор TRIAD

Деструктор TRIAD

Деструктор TRIAD

Деструктор TRIAD

Деструктор TRIAD

Ответы на вопросы

1. Наследование используется для создания нового класса на основе уже существующего, обеспечивая повторное использование кода и образуя иерархии «общее → частное».
2. public-компоненты базового класса становятся public в производном классе (доступны везде).
3. private-компоненты базового класса не наследуются в смысле доступа — они есть в объекте, но недоступны даже в производном классе.
4. protected-компоненты базового класса становятся protected в производном классе (доступны внутри самого класса и его наследников, но не снаружи).
5. Производный класс описывается так:
`class Derived : [public|protected|private] Base {
};`
6. Конструкторы не наследуются (кроме C++11 `using Base::Base;` — но это не полноценное наследование).
7. Деструкторы не наследуются.
8. Порядок конструирования: сначала базовый класс, потом компоненты-объекты, затем сам производный.
9. Порядок уничтожения: обратный конструированию: сначала производный класс, потом компоненты-объекты, затем базовый.
10. Виртуальные функции позволяют вызывать функцию производного класса через указатель/ссылку на базовый. Позднее связывание — выбор вызываемой функции во время выполнения (через VMT).
11. Конструкторы не могут быть виртуальными. Деструкторы могут и должны быть виртуальными в полиморфных иерархиях.
12. Да, спецификатор `virtual` наследуется (если функция виртуальна в базовом, она остаётся виртуальной во всех производных).
13. Открытое наследование (`public`) выражает отношение «is-a» (объект производного класса является частным случаем базового).
14. Закрытое наследование (`private`) выражает отношение «implemented-in-terms-of» (реализован с помощью), не является подтипом.
15. Принцип подстановки (Лисков): объект производного класса можно использовать везде, где ожидается объект базового, без нарушения корректности программы.
- 16.

Объект `Teacher` `x` будет включать (физически в памяти):

`Student::age` (приватный)

`Student::name`

`Employee::post`

`Teacher::stage`

Доступны из самого класса Teacher: name, post, stage.

age – недоступен, но существует.

```
class Student {
    int age;           // private – есть в объекте, но недоступен
public:
    string name;       // public
};
```

```
class Employee : public Student {
protected:
    string post;       // protected
};
```

```
class Teacher : public Employee {
protected:
    int stage;         // protected
};
```

17.

```
class Student {
    int age;
public:
    string name;
    Student() : age(0), name("") {}
};
```

```
class Employee : public Student {
protected:
    string post;
public:
    Employee() : Student(), post("") {}
};
```

```
class Teacher : public Employee {
protected:
    int stage;
public:
    Teacher() : Employee(), stage(0) {}
};
```

};

18.

```
class Student {
    int age;
public:
    string name;
    Student(int a, string n) : age(a), name(n) {}
};
```

```
class Employee : public Student {
protected:
    string post;
public:
    Employee(int a, string n, string p) : Student(a, n), post(p) {}
};
```

```
class Teacher : public Employee {
protected:
    int stage;
public:
    Teacher(int a, string n, string p, int s)
        : Employee(a, n, p), stage(s) {}
};
```

};

19.

```
class Student {
    int age;
```

```

public:
    string name;
    Student(const Student& other) : age(other.age), name(other.name) {}
};

class Employee : public Student {
protected:
    string post;
public:
    Employee(const Employee& other)
        : Student(other), post(other.post) {
    }
};

class Teacher : public Employee {
protected:
    int stage;
public:
    Teacher(const Teacher& other)
        : Employee(other), stage(other.stage) {
    }
};
20.
class Student {
    int age;
public:
    string name;
    Student& operator=(const Student& other) {
        if (this != &other) {
            age = other.age;
            name = other.name;
        }
        return *this;
    }
};

class Employee : public Student {
protected:
    string post;
public:
    Employee& operator=(const Employee& other) {
        if (this != &other) {
            Student::operator=(other);
            post = other.post;
        }
        return *this;
    }
};

class Teacher : public Employee {
protected:
    int stage;
public:
    Teacher& operator=(const Teacher& other) {
        if (this != &other) {
            Employee::operator=(other);
            stage = other.stage;
        }
        return *this;
    }
};

```

